

Introduction to Machine Learning Methods

Logistic regression

Haris Gačanin
RWTH Aachen University

Outline

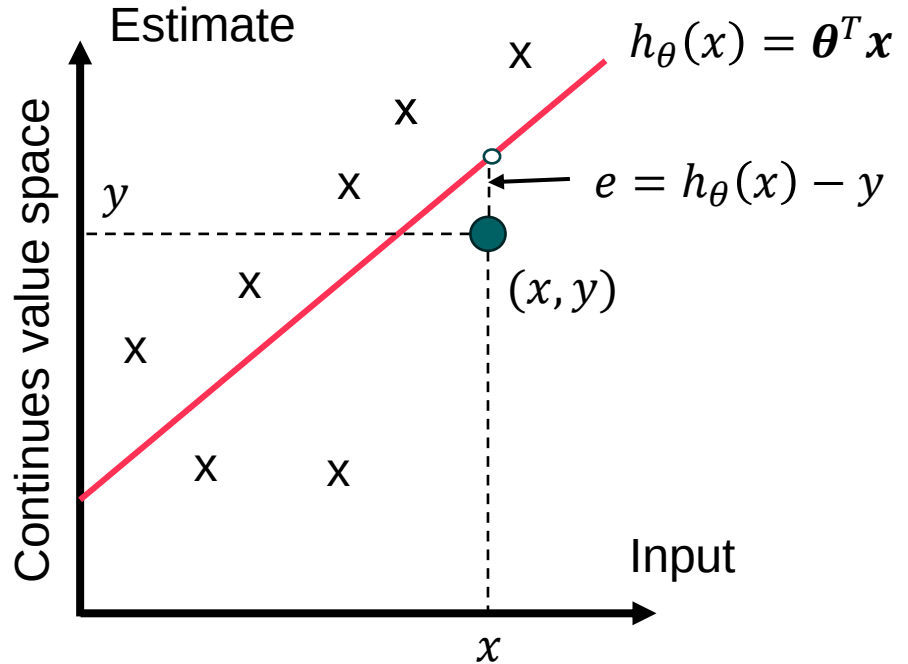
- Classification and hypothesis representation
- Loss function and gradient descent
- Binary vs multi-class classification
- Regularization for linear regression

Reference chapters: 18.4.3, 18.6.3, 18.6.4,

Classification and hypothesis representation

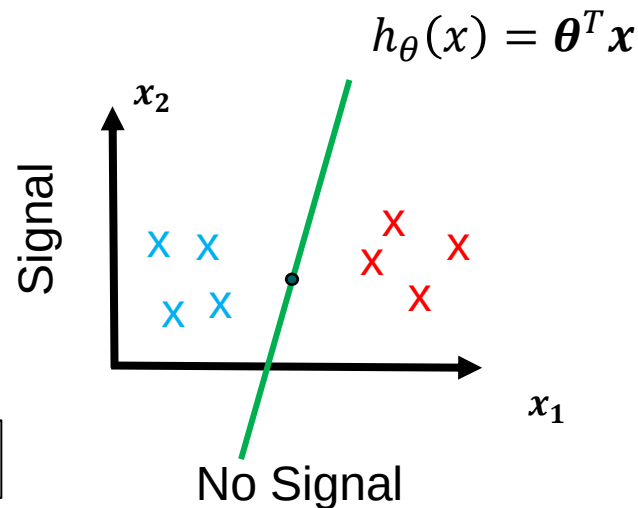
Linear regression

- Until now, we used a linear function for regression
 - Estimated variables have continuous values
- How about using linear function for classification?



Linear classifiers with a hard threshold

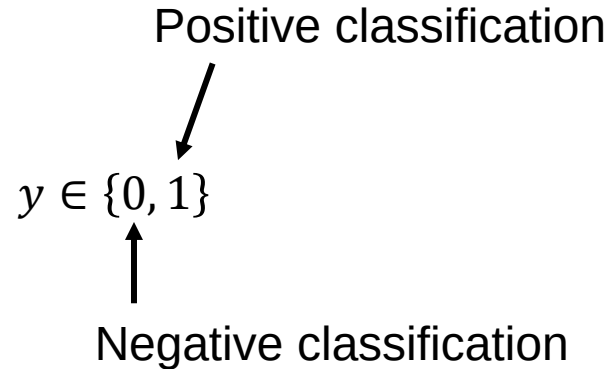
- A decision boundary is a straight line (or surface, in higher dimensions)
- Given the training data the task of classification is to learn a hypothesis h that takes new (x_1, x_2) points, and returns either 0 or 1 (binary output)



Two-class example

Classification examples

- Communication: Signal / No Signal
- Production: Success / Failure
- Health: Fever / No Fever



- Beyond binary classification is possible as well!



Regression vs. Classification model selection

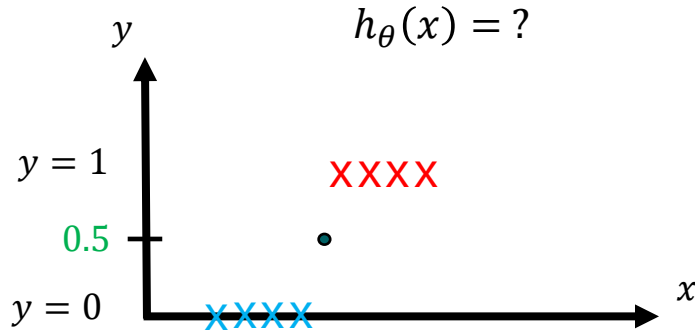
Linear regression – *continuous outputs (predicted)*

Classification problems – *discrete outputs (predicted)*

- Sometimes linear regression works well in classification problems

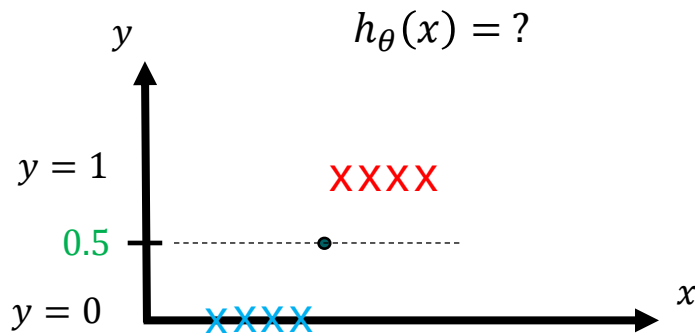
Classifier's threshold

- We consider designing a model with the following training (red, blue) examples



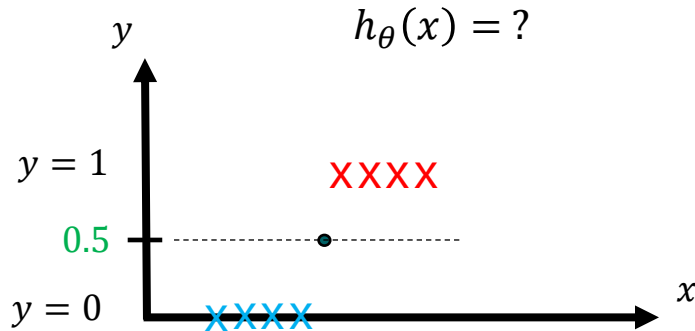
Classifier's threshold

- We consider designing a model with the following training (red, blue) examples
- For a given input x , the output is evaluated as follows:
 - $h_{\theta}(x) \geq 0.5$, predicts **positive** classification “ $y = 1$ ”
 - $h_{\theta}(x) < 0.5$, predicts negative classification “ $y = 0$ ”



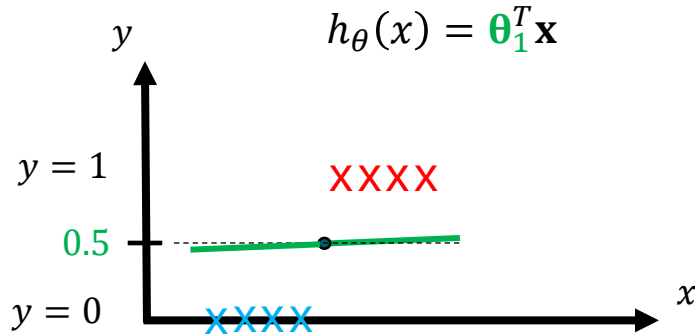
Classifier's threshold

- We consider designing a model with the following training (red, blue) examples
- For a given input x , the output is evaluated as follows:
 - $h_{\theta}(x) \geq 0.5$, predicts positive classification “ $y = 1$ ”
 - $h_{\theta}(x) < 0.5$, predicts negative classification “ $y = 0$ ”



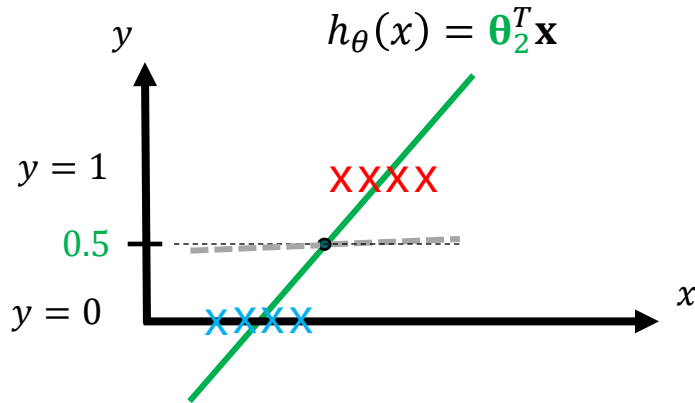
Model selection

- We consider designing a model with the following example
- For a given input x , the output is evaluated as follows:
 - $h_{\theta}(x) \geq 0.5$, predicts positive classification “ $y = 1$ ”
 - $h_{\theta}(x) < 0.5$, predicts negative classification “ $y = 0$ ”



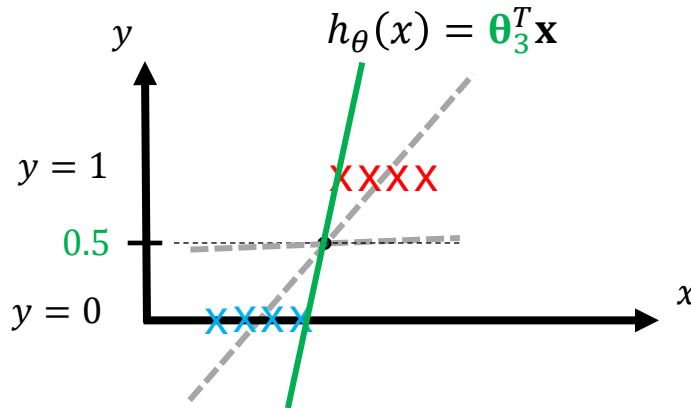
Model selection

- We consider designing a model with the following example
- For a given input x , the output is evaluated as follows:
 - $h_{\theta}(x) \geq 0.5$, predicts positive classification “ $y = 1$ ”
 - $h_{\theta}(x) < 0.5$, predicts negative classification “ $y = 0$ ”



Model selection

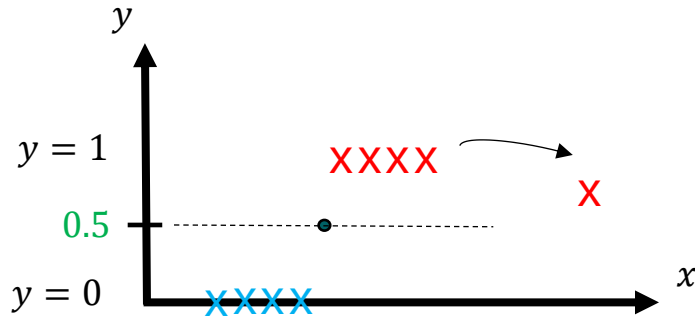
- We consider designing a model with the following example
- For a given input x , the output is evaluated as follows:
 - $h_{\theta}(x) \geq 0.5$, predicts positive classification “ $y = 1$ ”
 - $h_{\theta}(x) < 0.5$, predicts negative classification “ $y = 0$ ”



Until when can this approach work?

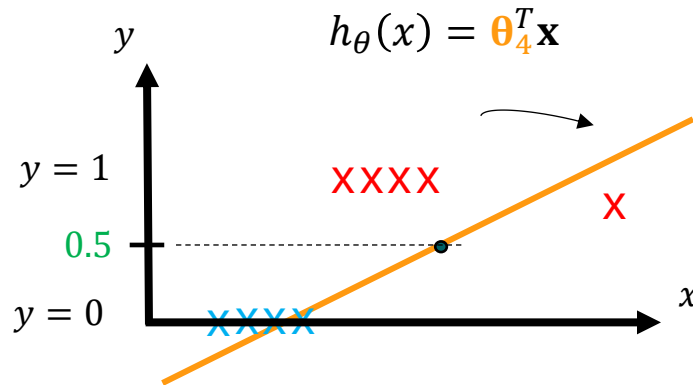
Why is the linear model not good for classification?

- Let's assume we add a new training example...



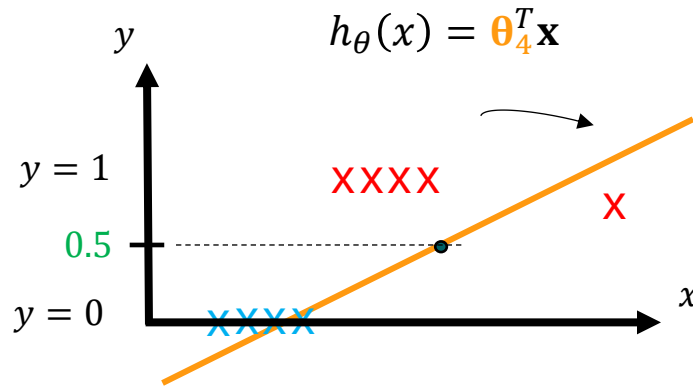
Why is the linear model not good for classification?

- Let's assume we add a new training example...
- The green model defined by θ_4 is retrained with the new training example



Why is the linear model not good for classification?

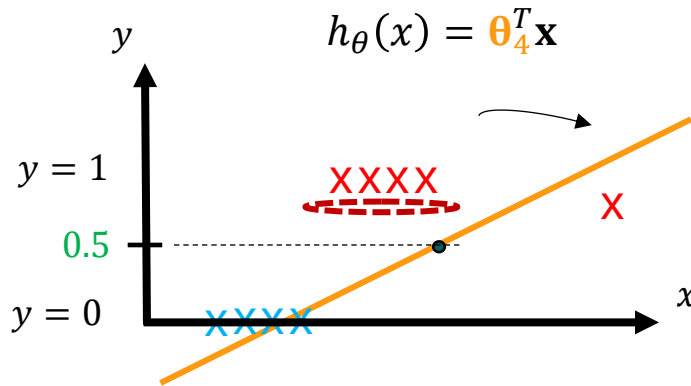
- Let's assume we add a new training example...
- The green model defined by θ_4 is retrained with the new training example



This new example is changing the slope of the model!

Why is the linear model not good for classification?

- Let's assume we add a new training example...
- The green model defined by θ_4 is retrained with the new training example



This new example is changing the slope of the model!

Consequently, this may lead to poor prediction!
(verify this in the example)



Design of a model for classification

Hypothesis for classification

- Hypothesis (discrete output) representation
 - $y = 0$ or $y = 1$
- Hypothesis is bounded as $0 \leq h_{\theta}(x) \leq 1$

Hypothesis for classification

- Hypothesis (discrete output) representation
 - $y = 0$ or $y = 1$
- Hypothesis is bounded as $0 \leq h_{\theta}(x) \leq 1$
- Instead of choosing a linear function for $h_{\theta}(x)$, now we select Sigmoid or Logistic function

$$h_{\theta}(x) = g(\theta^T \mathbf{x}) \quad \text{where} \quad g(x) = \frac{1}{1+e^{-x}}$$

Hypothesis for classification

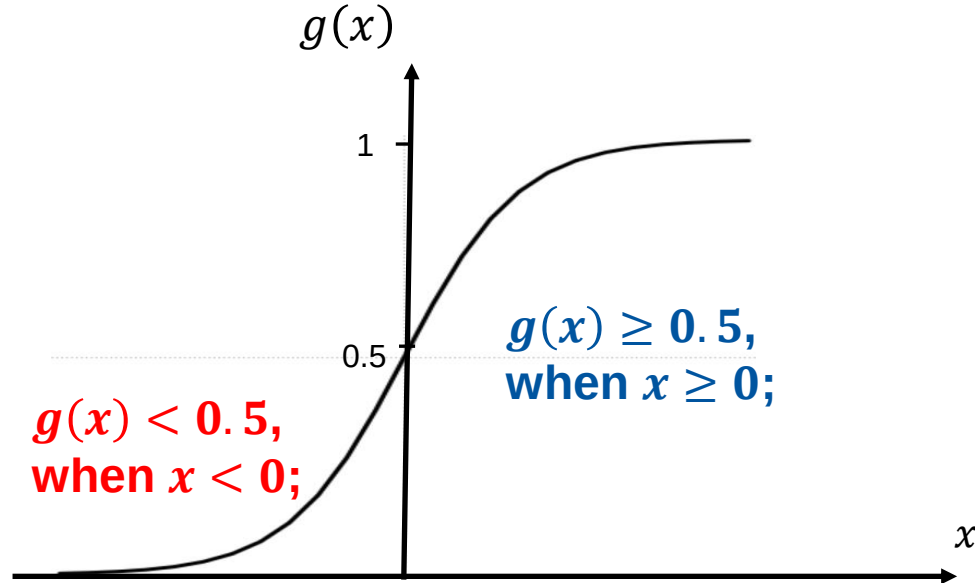
- Hypothesis (discrete output) representation
 - $y = 0$ or $y = 1$
- Hypothesis is bounded as $0 \leq h_{\theta}(x) \leq 1$
- Instead of choosing a linear function for $h_{\theta}(x)$, now we select Sigmoid or Logistic function

$$h_{\theta}(x) = g(\theta^T \mathbf{x}) \quad \text{where} \quad g(x) = \frac{1}{1+e^{-x}}$$

- The hypothesis $h_{\theta}(x)$ is represented by

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T \mathbf{x}}}$$

Sigmoid function



More about these functions in the next lecture...

Prediction with logistic regression

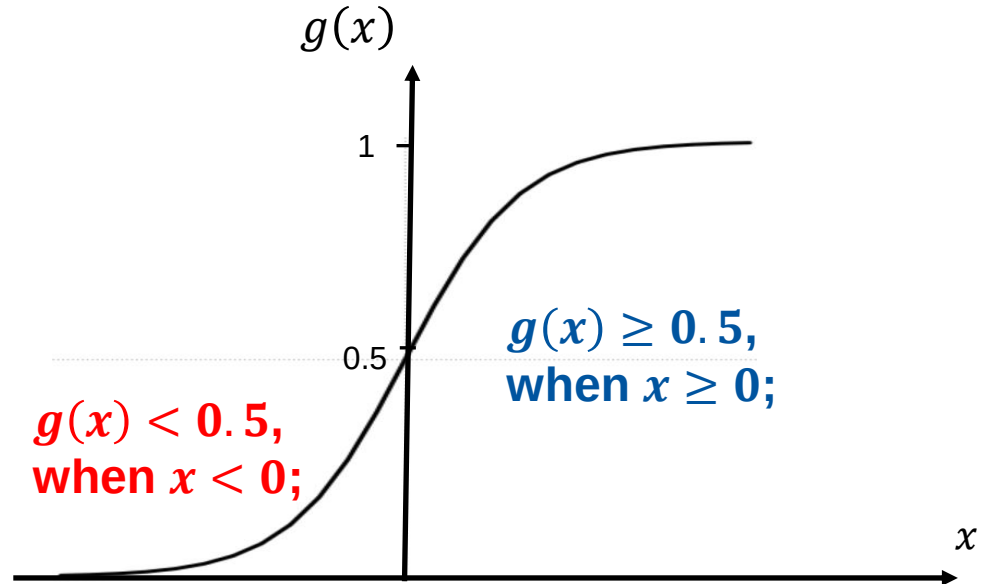
- Consider,

$$h_{\theta}(x) = g(\boldsymbol{\theta}^T \mathbf{x}) \quad \text{and} \quad g(x) = \frac{1}{1+e^{-x}}$$

- Suppose we want to predict

“ $y = 1$ ” if $h_{\theta}(x) \geq 0.5$

– This implies $\boldsymbol{\theta}^T \mathbf{x} \geq 0$



Prediction with logistic regression

- Consider,

$$h_{\theta}(x) = g(\theta^T \mathbf{x}) \quad \text{and} \quad g(x) = \frac{1}{1+e^{-x}}$$

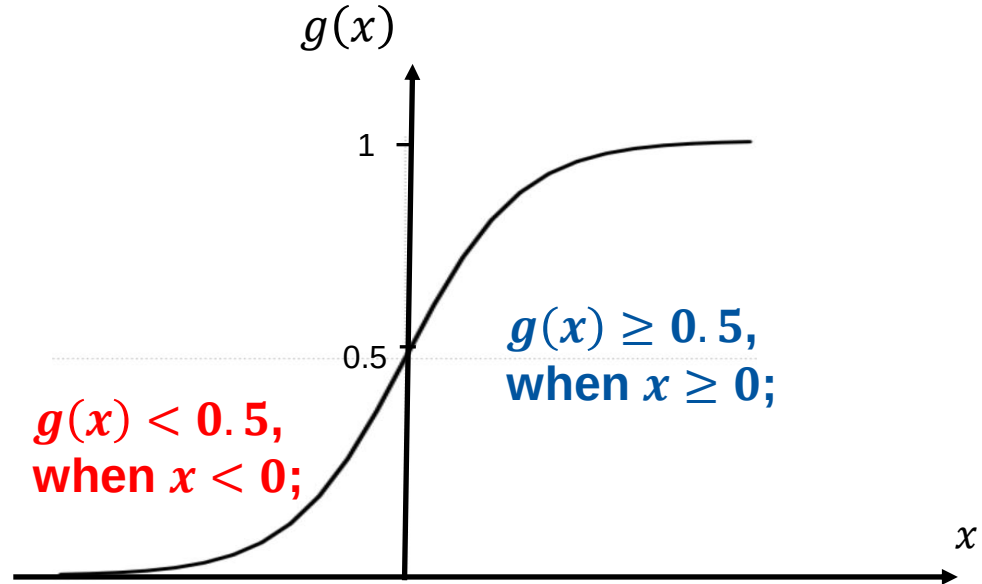
- Suppose we want to predict

“ $y = 1$ ” if $h_{\theta}(x) \geq 0.5$

– This implies $\theta^T \mathbf{x} \geq 0$

“ $y = 0$ ” if $h_{\theta}(x) < 0.5$

– This implies $\theta^T \mathbf{x} < 0$



Prediction with logistic regression

- Consider,

$$h_{\theta}(x) = g(\theta^T \mathbf{x}) \quad \text{and} \quad g(x) = \frac{1}{1+e^{-x}}$$

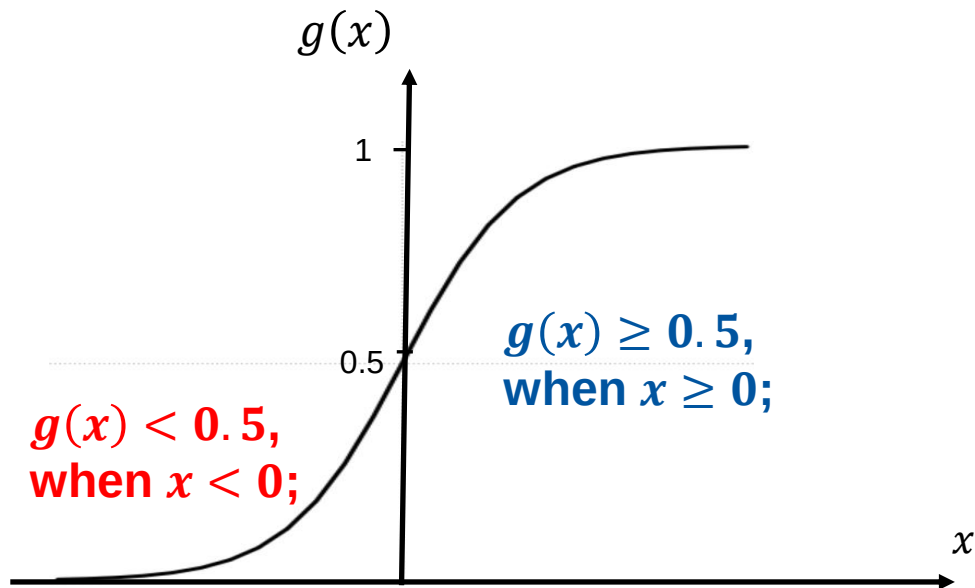
- Suppose we want to predict

“ $y = 1$ ” if $h_{\theta}(x) \geq 0.5$

– This implies $\theta^T \mathbf{x} \geq 0$

“ $y = 0$ ” if $h_{\theta}(x) < 0.5$

– This implies $\theta^T \mathbf{x} < 0$



Model defines the probability that $y = 1$ given the input x and parametrized by θ

$$h_{\theta}(x) = P(y = 1|x; \theta)$$



Linear vs. nonlinear decision boundaries

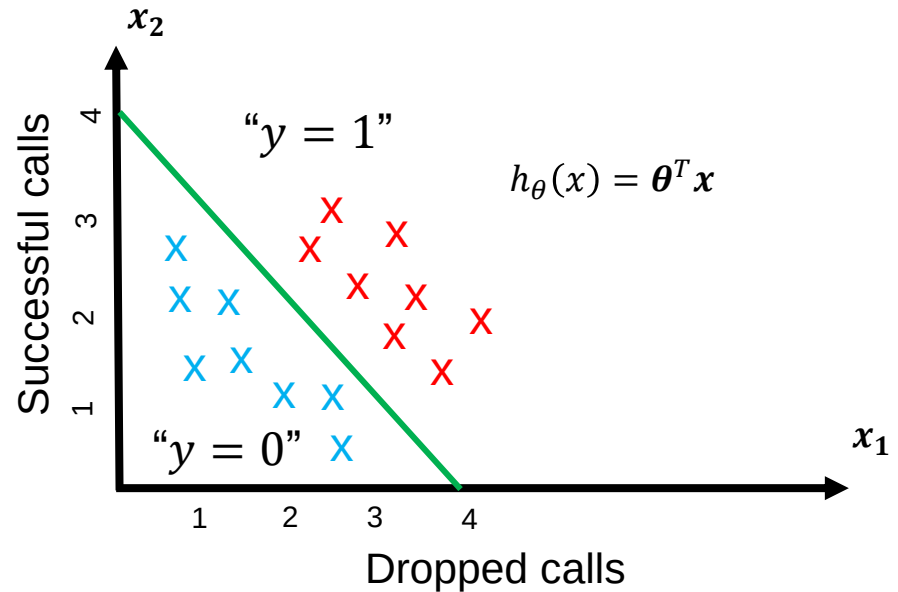
Linear decision boundary

- $h_{\theta}(x) = g(\theta^T \mathbf{x}) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$

- Example $\theta = [-4 \ 1 \ 1]^T$

- Predict “ $y = 1$ ” if $x_1 + x_2 \geq 4$

- Predict “ $y = 0$ ” if $x_1 + x_2 < 4$



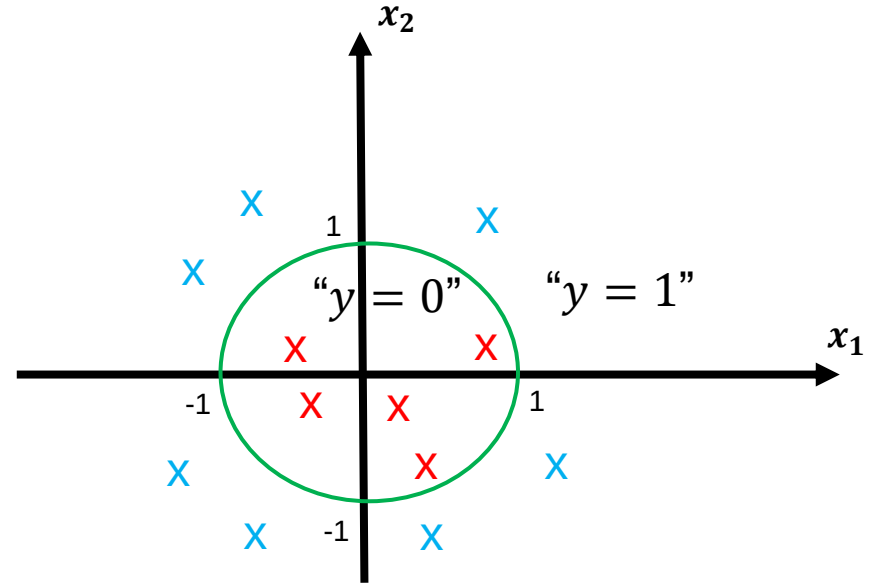
Nonlinear decision boundary

- $h_{\theta}(x) = g(\theta^T \mathbf{x}) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$

- Example $\theta = [-1 \ 0 \ 0 \ 1 \ 1]^T$

- Predict “ $y = 1$ ” if $x_1^2 + x_2^2 \geq 1$

- Predict “ $y = 0$ ” if $x_1^2 + x_2^2 < 1$



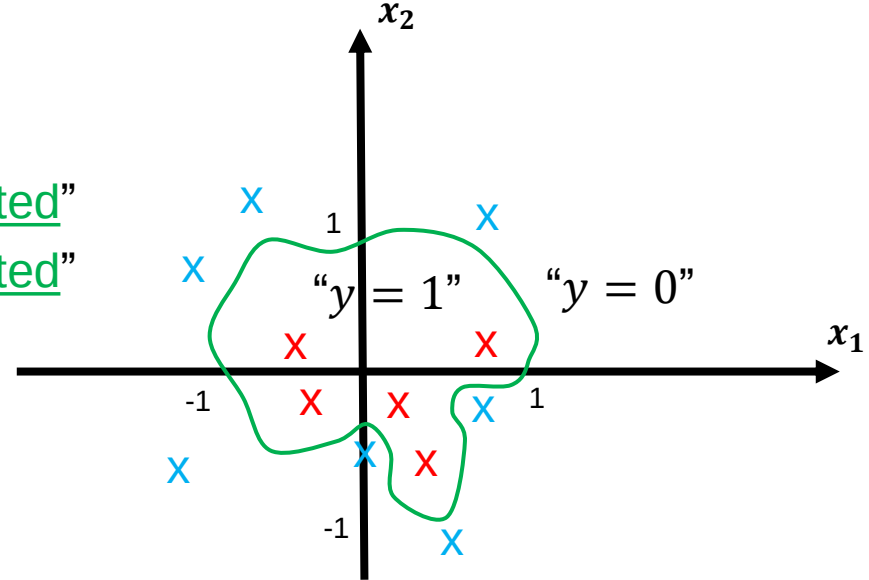
Nonlinear decision boundary

- $h_{\theta}(x) = g(\theta^T x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^2 x_2^2 + \dots)$

- Example $\theta = [\dots]^T$

- Predict “ $y = 0$ ” if “now is more complicated”

- Predict “ $y = 1$ ” if “now is more complicated”



complex decision boundary



Cost function for logistic regression

The cost function is defined as the mean of the loss function across all training examples

The loss function is defined for one training example

Definitions

- Assume a classification problem $y \in \{0,1\}$ with m training examples $(x^{(i)}, y^{(i)})$

Definitions

- Assume a classification problem $y \in \{0,1\}$ with m training examples $(x^{(i)}, y^{(i)})$
- Logistic regression hypothesis:

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T \mathbf{x}}}$$

where $\mathbf{x} \in R^{n+1}$ ($x_0 = 1$)

Definitions

- Assume a classification problem $y \in \{0,1\}$ with m training examples $(x^{(i)}, y^{(i)})$
- Logistic regression hypothesis:

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T \mathbf{x}}}$$

where $\mathbf{x} \in R^{n+1}$ ($x_0 = 1$)

- **Problem**: The cost function for linear regression is not suitable

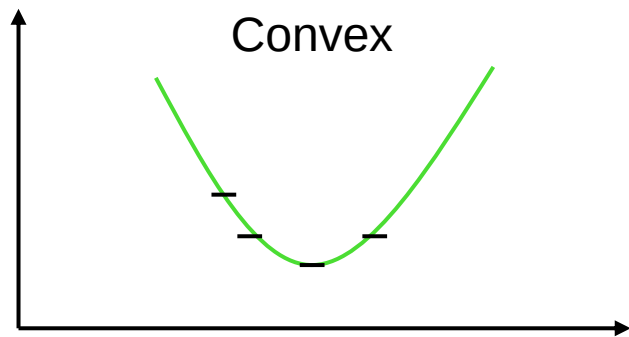
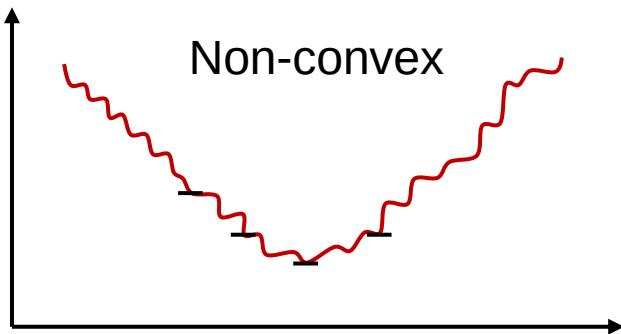
$$\begin{aligned} J(\boldsymbol{\theta}) &= \frac{1}{2m} \sum_{i=1}^m [h_{\boldsymbol{\theta}}(x^{(i)}) - y^{(i)}]^2 \\ &= \frac{1}{2m} \sum_{i=1}^m \left[\frac{1}{1 + e^{-\boldsymbol{\theta}^T \mathbf{x}}} - y^{(i)} \right]^2 \end{aligned}$$

Remarks

- The linear regression cost function with sigmoid predictor

$$J(\boldsymbol{\theta}) = \frac{1}{2m} \sum_{i=1}^m \left[\frac{1}{1 + e^{-\boldsymbol{\theta}^T \mathbf{x}}} - y^{(i)} \right]^2$$

- Is a non-convex function
- Because of sigmoid term the loss function is nonlinear
- No guarantee to converge to global minimum



We need to define a convex loss function with sigmoid function

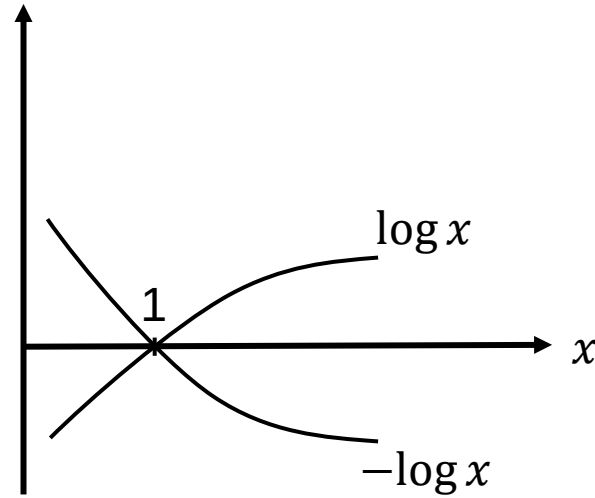
- We search for a convex loss function $Loss(h_{\theta}(x^{(i)}), y)$

$$J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} [h_{\theta}(x^{(i)}) - y^{(i)}]^2 \Rightarrow \frac{1}{m} \sum_{i=1}^m Loss(h_{\theta}(x^{(i)}), y)$$

so we can apply gradient descent to search for the minimum!

Examples of convex functions

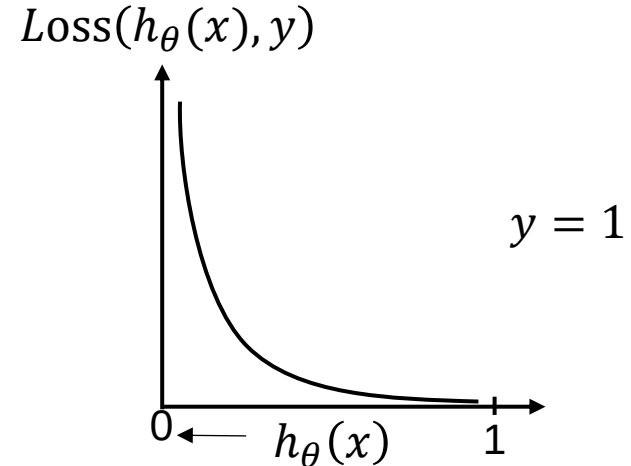
- Examples: Logarithmic functions!



Properties of log functions

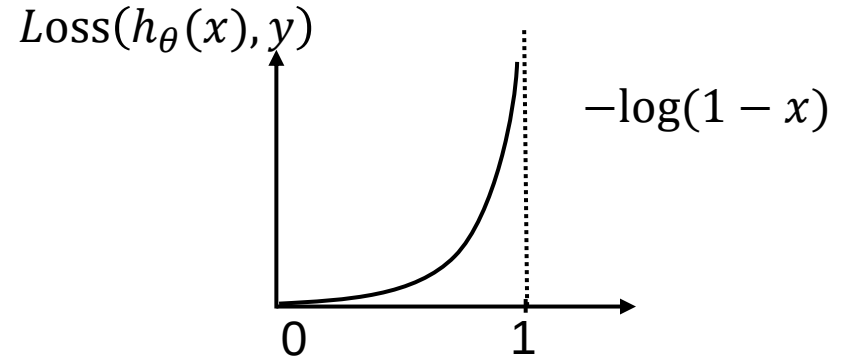
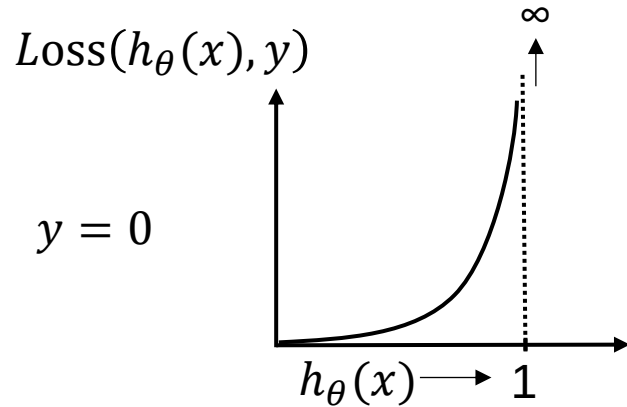
$$\text{Loss}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)); & y = 1 \\ -\log(1 - h_{\theta}(x)); & y = 0 \end{cases}$$

- When $y = 1$ and $h_{\theta}(x) = 1$
– $\text{Loss}(h_{\theta}(x), y) = 0$
- When $h_{\theta}(x) \rightarrow 0$
– $\text{Loss}(h_{\theta}(x), y) \rightarrow \infty$ and predicts
 $P(y = 1|x; \theta) = 0$, will penalize learning
algorithm by a very large cost.



Properties of log functions

$$L(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)); & y = 1 \\ -\log(1 - h_{\theta}(x)); & y = 0 \end{cases}$$



Cost and Loss functions

- Suppose we want to classify “ $y = 1$ ” or “ $y = 0$ ”

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Loss}(h_{\theta}(x^{(i)}), y^{(i)})$$

where the cost function is given by

$$\text{Loss}(h_{\theta}(x^{(i)}), y^{(i)}) = \begin{cases} -\log[h_{\theta}(x^{(i)})], & \text{if } y = 1 \\ -\log[1 - h_{\theta}(x^{(i)})], & \text{if } y = 0 \end{cases}$$

Loss function in compact form

- From the previous definition of loss function, we have

$$Loss(h_{\theta}(x), y) = -y \log(h_{\theta}(x)) - (1 - y) \log(1 - h_{\theta}(x))$$

– If $y = 1$

$$Loss(h_{\theta}(x), 1) = -\log(h_{\theta}(x)) - 0$$

– If $y = 0$

$$Loss(h_{\theta}(x), 0) = 0 - \log(1 - h_{\theta}(x))$$

Logistic regression cost function

- Finally, the cost function of logistic regression can be expressed as:

$$\begin{aligned} J(\theta) &= \frac{1}{m} \sum_{i=1}^m \text{Loss}(h_{\theta}(x^{(i)}), y^{(i)}) \\ &= -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))] \end{aligned}$$



Parameter update

Minimization of the cost function

- Given the cost function for logistic regression, we calculate parameters θ by finding

$$\min_{\theta} J(\theta) = \min_{\theta} \left\{ -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))] \right\}$$

- Remember the method “1,2,3,4” from previous lecture!

- Note: $h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$

Update of parameters with gradient descent

- Repeat {

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial J(\boldsymbol{\theta})}{\partial \theta_j}$$

} Simultaneously update $\boldsymbol{\theta}$

- For logistic regression, $h_{\theta} = \frac{1}{1+e^{-\boldsymbol{\theta}^T \mathbf{x}}}$ (or $h_{\theta} = \boldsymbol{\theta}^T \mathbf{x}$ for linear regression)
 - Algorithm is identical as linear regression
 - Regularization is relevant for logistic regression



Binary vs multi-class classification

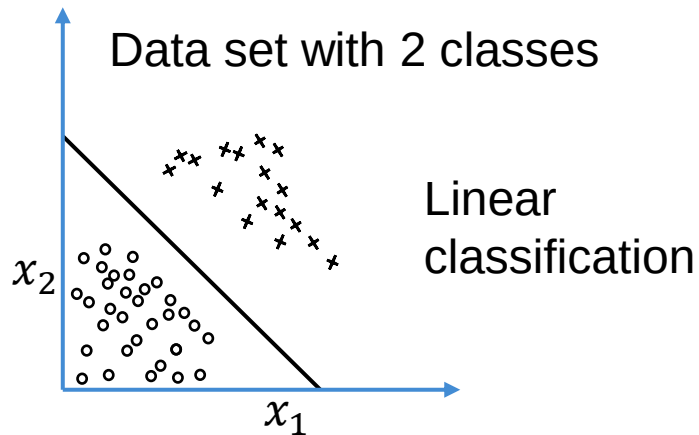
Binary vs multi-class classification

- Consider a problem of multi-class classification:
 - Animal classification: Elephant ($y = 1$), Tiger ($y = 2$), Mouse ($y = 3$), Bird ($y = 4$)
 - Signal demodulation: QPSK ($y = 1$), BPSK ($y = 2$), 16-QAM ($y = 3$)
 - Text/number recognition: a/0 ($y = 1$), b/1 ($y = 2$), c/2 ($y = 3$), d/3 ($y = 4$).

Binary vs multi-class classification

- Consider a problem of multi-class classification:
 - Animal classification: Elephant ($y = 1$), Tiger ($y = 2$), Mouse ($y = 3$), Bird ($y = 4$)
 - Signal demodulation: QPSK ($y = 1$), BPSK ($y = 2$), 16-QAM ($y = 3$)
 - Text/number recognition: a/0 ($y = 1$), b/1 ($y = 2$), c/2 ($y = 3$), d/3 ($y = 4$).

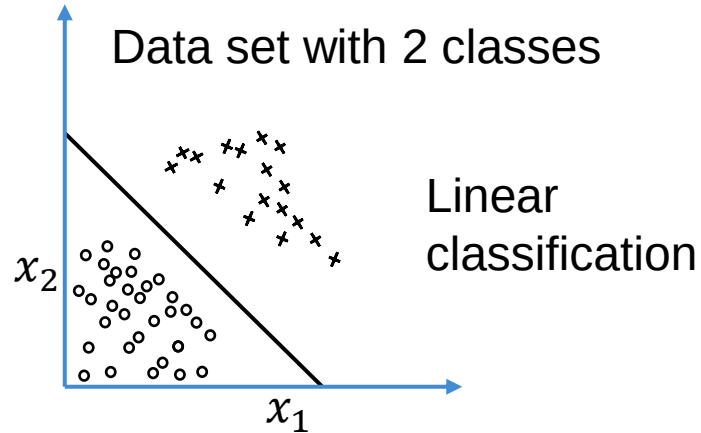
Binary classification



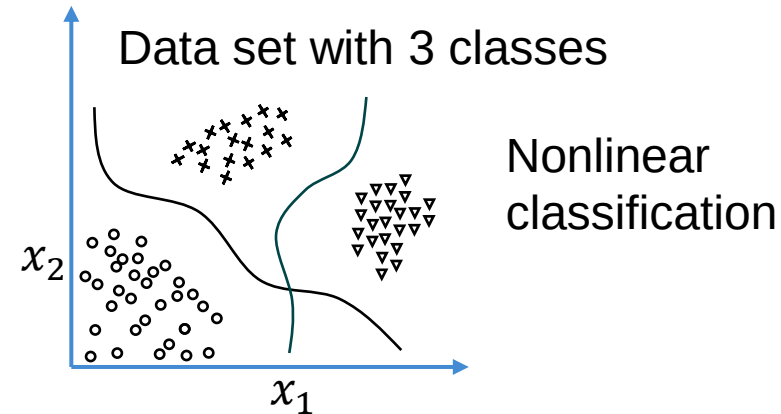
Binary vs multi-class classification

- Consider a problem of multi-class classification:
 - Animal classification: Elephant ($y = 1$), Tiger ($y = 2$), Mouse ($y = 3$), Bird ($y = 4$)
 - Signal demodulation: QPSK ($y = 1$), BPSK ($y = 2$), 16-QAM ($y = 3$)
 - Text/number recognition: a/0 ($y = 1$), b/1 ($y = 2$), c/2 ($y = 3$), d/3 ($y = 4$)

Binary classification



Multi-class classification



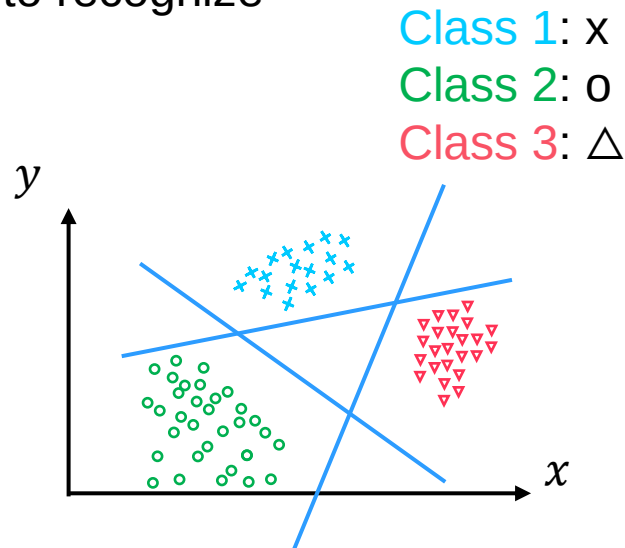
Multi-class logistic regression: One-vs-All

- Take the training set and turn this into three separate two-class problems.
 - Class 1 is positive; Class 2 and Class 3 are negative
- Results are three classifiers for $i = 1, 2, 3$ each trained to recognize one of the three classes as:

$$h_{\theta}^{(i)}(x) = P(y = i | x; \theta)$$

- On a new input x , pick the class i to maximize

$$\max_i h_{\theta}^{(i)}(x)$$



Multi-class cost function with L_2 regularization

- Recall the cost of the logistic regression function, given by:

$$\begin{aligned} J(\theta) &= -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \left(P \left(y^{(i)} = k | x^{(i)} \right) \right) + \frac{\lambda}{2m} \sum_{k=1}^K \sum_{j=1}^n (\theta_{kj})^2 \\ &= -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \left(h_{\theta}(x^{(i)})_k \right) + \frac{\lambda}{2m} \sum_{k=1}^K \sum_{j=1}^n (\theta_{kj})^2 \end{aligned}$$

where the activation is given by softmax probability as

$$P \left(y^{(i)} = k | x^{(i)} \right) = h_{\theta}(x^{(i)})_k = \frac{e^{\theta_k^T \mathbf{x}^{(i)}}}{\sum_{j=1}^K e^{\theta_j^T \mathbf{x}^{(i)}}}$$

Note: bias term θ_0 is NOT regularized



Regularization of cost function

Chapter 18.4.3

Recap: overfitting in linear regression

- With many features, the learned $h_{\theta}(x)$ fits well the training set or

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Loss}(h_{\theta}(x^{(i)}), y^{(i)}) \approx 0$$

- Thus, when $J(\theta) \approx 0$, $h_{\theta}(x)$ fails to generalize (i.e., predict) on unseen data!

Recap: overfitting in linear regression

- With many features, the learned $h_{\theta}(x)$ fits well the training set or

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Loss}(h_{\theta}(x^{(i)}), y^{(i)}) \approx 0$$

- Thus, when $J(\theta) \approx 0$, $h_{\theta}(x)$ fails to generalize (i.e., predict) on unseen data!

Recap: overfitting in linear regression

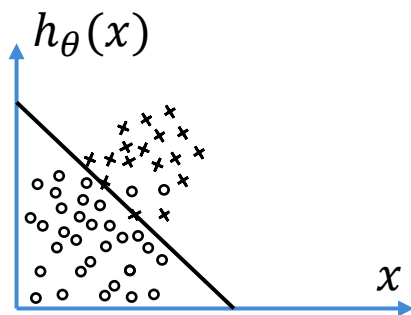
- With many features, the learned $h_{\theta}(x)$ fits well the training set or

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Loss}(h_{\theta}(x^{(i)}), y^{(i)}) \approx 0$$

- Thus, when $J(\theta) \approx 0$, $h_{\theta}(x)$ fails to generalize (i.e., predict) on unseen data!

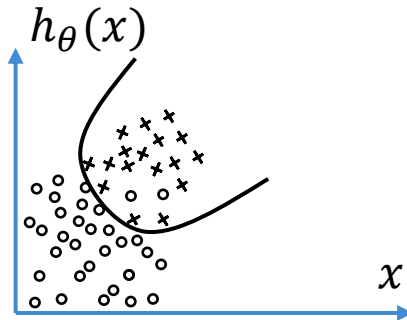
How about overfitting for logistic regression?

Underfitting, **high bias**



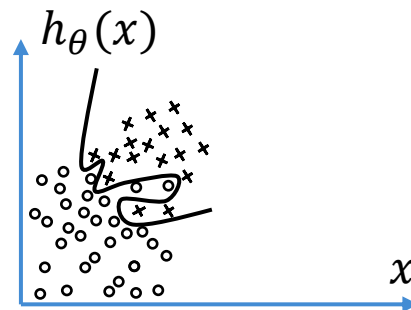
$$h_\theta(x) = g(\theta_0 + \theta_1 x_1)$$

Good fit



$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$$

Overfitting, **high variance**

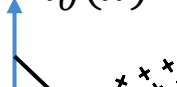


$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2 + \theta_6 x_1^3 x_2 + \dots)$$

How about overfitting for logistic regression?

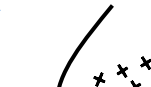
Underfitting, **high bias**

$h_{\theta}(x)$



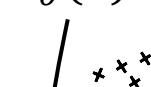
Good fit

$h_{\theta}(x)$



Overfitting, **high variance**

$h_{\theta}(x)$



- $h_{\theta}(x)$ learns the “training data” and not the (generator) function f
- Performs well on the seen examples, but poorly with new data

$$+ \theta_3 x_1 + \theta_4 x_2 + \theta_5 x_1 x_2)$$

$$+ \theta_3 x_1 + \theta_4 x_2 + \theta_5 x_1 x_2 \\ + \theta_6 x_1^3 x_2 + \dots)$$

Remark

- Either the loss function be linear or logistic:

$$\text{Loss}(h_{\theta}(x^{(i)}), y^{(i)}) = \begin{cases} [h_{\theta}(x^{(i)}) - y^{(i)}]^2 & \text{linear reg.} \\ -[y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)}))] & \text{logistic reg.} \end{cases}$$

the regularization principle does not change (applies the same).

How to overcome the overfitting problem?

Option 1: Reduce the number of input predictors (features) $\mathbf{x}$

- Manually (based on heuristics, experience) select which predictors to keep
- The negative side is the loss of information due to a reduction in the number of predictors $\mathbf{x}$

$$\min_{\theta} \frac{1}{m} \sum_{i=1}^m \text{Loss}(h_{\theta}(\mathbf{x}^{(i)}), y^{(i)})$$

How to overcome the overfitting problem?

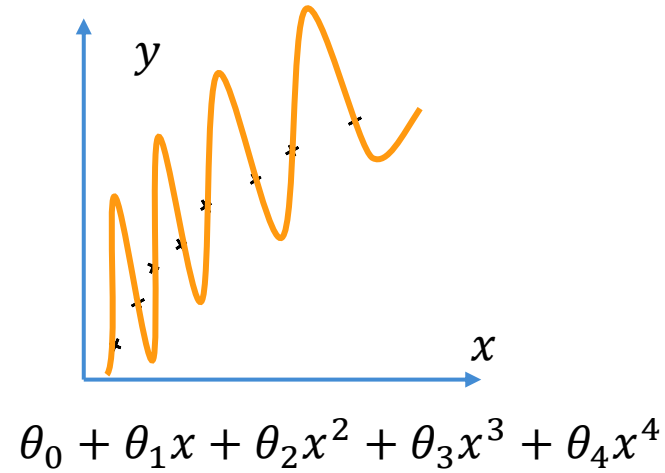
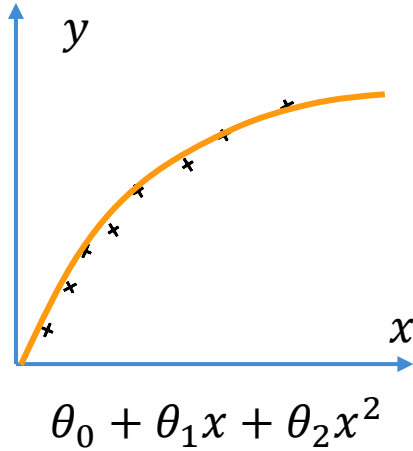
Option 2: Regularization

- Keep all the predictors, but reduce the magnitudes of the parameters θ
- Good option when we have a lot of predictors, each of which contributes a bit to predicting y

$$\min_{\theta} \frac{1}{m} \sum_{i=1}^m \text{Loss}(h_{\theta}(x^{(i)}), y^{(i)})$$

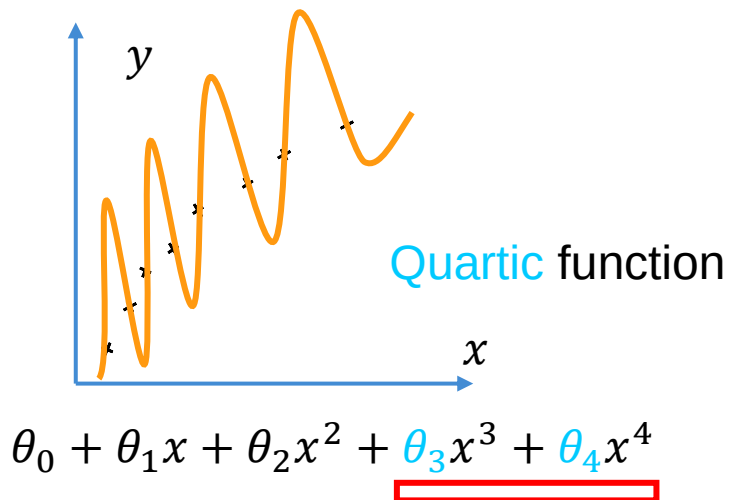
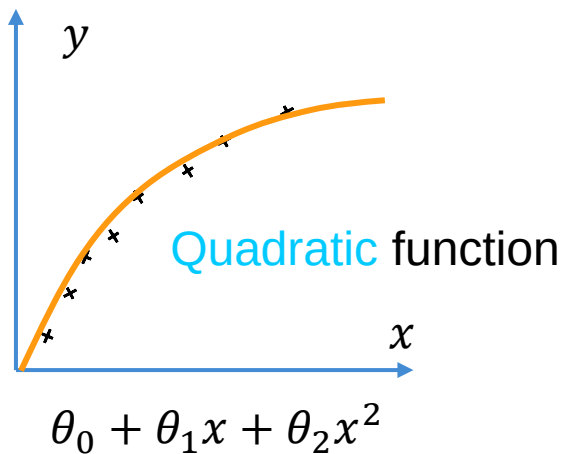
An example

- Let's consider the following functions



An example

- Let's consider the following functions

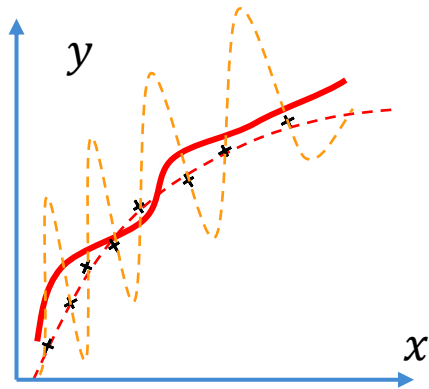


- Next, we regularize the cost function by making θ_3 and θ_4 very small

$$(\theta_3 \approx 0, \theta_4 \approx 0)$$

A function regularization principle

$$\min_{\theta} \frac{1}{2m} \left\{ \sum_{i=1}^m [h_{\theta}(x^{(i)}) - y^{(i)}]^2 + 10^3(\theta_3^2 + \theta_4^2) \right\}$$

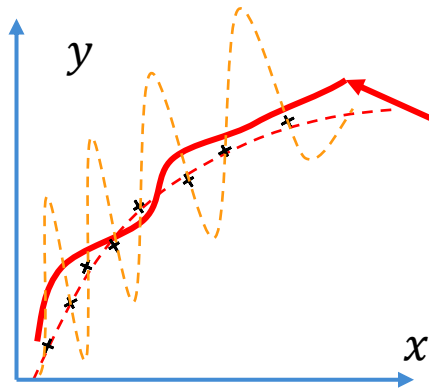


$$\theta_0 + \theta_1 x + \theta_2 x^2 + \cancel{\theta_3 x^3} + \cancel{\theta_4 x^4}$$

$\approx 0 \quad \approx 0$

From complex to simpler models by reducing θ

$$\min_{\theta} \frac{1}{2m} \left\{ \sum_{i=1}^m [h_{\theta}(x^{(i)}) - y^{(i)}]^2 + 10^3(\theta_3^2 + \theta_4^2) \right\}$$



Due to the second term, the model moves away from the quartic function and approaches the quadratic function (but we never get the exact quadratic function)

$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

$\approx 0 \quad \approx 0$

Regularization principle

- Add a regularization term as:

regularization term ($j \geq 1$)

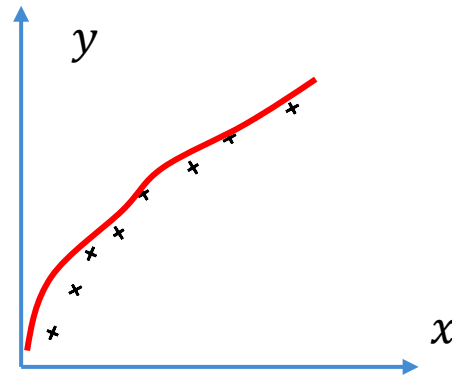
$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

- Regularization parameter λ controls the trade-off between:
 - Fitting training set well (captured by the first term)
 - Keep the parameters small
 - If λ is selected very large, the algorithm results in ‘under fitting’ and fails to fit the training set.

Example of a model with a large λ

- What happens for large λ (e.g., 10^{10}) in:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$



Example of a model with a large λ

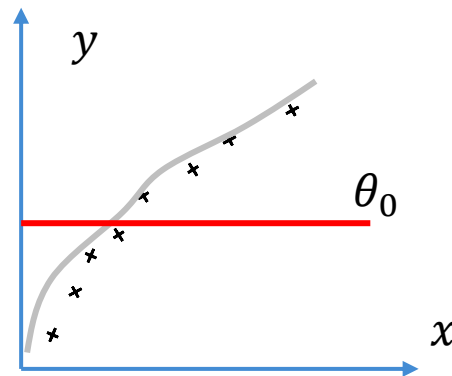
- What happens for large λ (e.g., 10^{10}) in:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

- For example, if we assume

$$\theta_1 \approx 0, \theta_2 \approx 0, \theta_3 \approx 0, \theta_4 \approx 0$$

$$h_{\theta}(x) = \theta_0 + \cancel{\theta_1 x} + \cancel{\theta_2 x^2} + \cancel{\theta_3 x^3} + \cancel{\theta_4 x^4}$$
$$\approx \theta_0 \Rightarrow \text{constant}$$



Example of a model with a large λ

- What happens for large λ (e.g., 10^{10}) in:

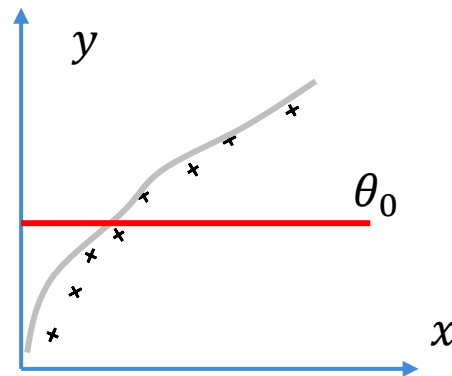
$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

- For example, if we assume

$$\theta_1 \approx 0, \theta_2 \approx 0, \theta_3 \approx 0, \theta_4 \approx 0$$

$$h_{\theta}(x) = \theta_0 + \cancel{\theta_1 x} + \cancel{\theta_2 x^2} + \cancel{\theta_3 x^3} + \cancel{\theta_4 x^4}$$
$$\approx \theta_0 \Rightarrow \text{constant}$$

- Algorithm fails to fit a training examples (underfitting)
- Gradient descent will not converge



Regularized linear regression

To minimize the cost function, we use the gradient descent algorithm as follows:

$$\begin{cases} \theta_0 \leftarrow \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)} \\ \theta_j \leftarrow \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j \right] \end{cases}$$

for $j = 1, 2, \dots, n$,

Remark

$$\theta_j \leftarrow \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j \right]$$

Remark

$$\theta_j \leftarrow \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j \right]$$

$$= \theta_j - \frac{\lambda}{m} \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Remark

$$\theta_j \leftarrow \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j \right]$$

$$= \theta_j - \frac{\lambda}{m} \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$= \underbrace{\theta_j \left(1 - \alpha \frac{\lambda}{m}\right)}_{(< 1) \approx 0.99} - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

$(< 1) \approx 0.99$

Reduction of the parameter value

$$\frac{\partial J(\theta)}{\partial \theta_j}$$

Final remark

- Logistic regression defines a probability of an event!
- Same derivation is used for logistic regression...Try it our at home!

Thank you!

